

AI ASSIATED CODING

ASSIGNMENT : 13.2

PROGRAM : B.TECH\CSE
SPECIALISATION : AIML
NAME : MANDALA.TEJASRI
ENROLMENT NUMBER : 2403A51171
COURSE : AI ASSISTED CODING
DATE : 29-10-2025
BATCH NO : 01

TASK DESCRIPTION#1

>> use gemini to generate a python code to Provide AI with the following redundant code and ask it to refactor

Python Code

```
def calculate_area(shape, x, y=0):  
if shape == "rectangle":  
return x * y  
elif shape == "square":  
return x * x  
elif shape == "circle":  
return 3.14 * x * x
```

PROMPT :

>> Develop a code to generate a python code to Provide AI with the following redundant code and ask it to refactor

EXPECTED OUTPUT AND SCREENSHOT :

```
code_to_refactor = """
def calculate_area(shape, x, y=0):
    if shape == "rectangle":
        return x * y
    elif shape == "square":
        return x * x
    elif shape == "circle":
        return 3.14 * x * x
"""

prompt = f"Refactor the following Python code:\n{code_to_refactor}"

response = gemini_model.generate_content(prompt)
print("Refactored Code:")
print(response.text)
```

OBSERVATION :

Based on the execution output, the code in cell `e95d5cbd` failed because the `GOOGLE_API_KEY` secret was not found in the Colab secrets manager. This means the program could not access your API key to configure the Gemini API.

TASK DESCRIPTION#2

>> use gemini to generate a python code to Legacy function without proper error handling

Python Code

```
def read_file(filename):
    f = open(filename, "r")
    data = f.read()

    f.close()
    return data
```

PROMPT : Develop a code to generate a python code to Legacy function without proper error handling

EXPECTED OUTPUT AND SCREENSHOT :

```
def read_file(filename):  
    """  
    Reads the content of a file with proper error handling.  
  
    Args:  
        filename: The name of the file to read.  
  
    Returns:  
        The content of the file, or None if an error occurred.  
    """
```

```
try:  
    with open(filename, "r") as f:  
        data = f.read()  
    return data  
except FileNotFoundError:  
    print(f"Error: File '{filename}' not found.")  
    return None  
except IOError:  
    print(f"Error: Could not read file '{filename}'.")  
    return None
```

OBSERVATION :

Based on the execution results of the visible cells:

- Cell `c2ae4789` successfully initialized the Gemini model.
- Cell `ebbb9bea` failed with a `DefaultCredentialsError` because the `GOOGLE_API_KEY` was not found, preventing the use of the Gemini model for refactoring.

Task Description #3

>>Use google gemini to Provide this legacy class to AI for readability and modularity

improvements:

Python Code

class Student:

def __init__(self, n, a, m1, m2, m3):

```
self.n = n
self.a = a
self.m1 = m1
self.m2 = m2
self.m3 = m3
def details(self):
print("Name:", self.n, "Age:", self.a)
def total(self):
return self.m1+self.m2+self.m3
```

PROMPT :

>> Develop a code to generate a python code to Provide this legacy class to AI for readability and modularity improvements:

EXPECTED OUTPUT AND SCREENSHOT :

```
legacy_class_definition = """
class Student:
    def __init__(self, n, a, m1, m2, m3):
        self.n = n
        self.a = a
        self.m1 = m1
        self.m2 = m2
        self.m3 = m3
    def details(self):
        print("Name:", self.n, "Age:", self.a)
    def total(self):
        return self.m1+self.m2+self.m3

prompt = f"Please refactor the following Python class for improved readability and modularity:\n\n{legacy_class_definition}"
response = gemini_model.generate_content(prompt)
print(response.text)
```

OBSERVATION :

It seems there was an error because the `GOOGLE_API_KEY` secret was not found. Please make sure you have added your API key to the Colab secrets manager under the name `GOOGLE_API_KEY`. After adding the key, you can run the code cell again.

Task Description #4

Generate a python code to Refactor this inefficient loop

Python Code

```
nums = [1,2,3,4,5,6,7,8,9,10]
squares = []
for i in nums:
    squares.append(i * i)
```

PROMPT :

>> Develop a code to generate a python code to Refactor this inefficient loop

EXPECTED OUTPUT AND SCREENSHOT :

```
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
squares = [i * i for i in nums]
```

OBSERVATION :

The list comprehension is preferred in Python for this type of operation because it's **faster** and more **concise** than the explicit `for` loop with `.append()`.

- **Performance:** In the standard CPython interpreter, list comprehensions are optimized to perform the list creation at the C level. They construct the entire list in one go, which is generally more efficient than the original method's repeated overhead of **method lookups** and **function calls** to `squares.append()` in every loop iteration.